

Chapter 49

USB 2.0 High-Speed OTG

49.1 Overview

The ESP32-P4 chip features a USB 2.0 High-Speed On-The-Go peripheral (OTG_HS) with an integrated transceiver. This OTG_HS complies with the USB 2.0 specification, OTG Revision 1.3, and OTG Revision 2.0 specifications. The interface supports USB 2.0 High-Speed mode (480 Mbit/s), Full-Speed mode (12 Mbit/s), and Low-Speed mode (1.5 Mbit/s).

- When OTG_HS operates in High-Speed or Full-Speed modes, it can be configured as either a Host or a Device.
- When OTG_HS operates in Low-Speed mode, it can only be configured as a Host.

49.2 Glossary

The following abbreviations and terms are used in this chapter.

Host	The host computer system where the USB Host Controller is installed. This includes the host hardware platform (CPU, bus, etc.) and the operating system in use.
Device	A logical or physical entity that performs a function. The actual entity described depends on the context of the reference. At the lowest level, device may refer to a single hardware component, as in a memory device. At a higher level, it may refer to a collection of hardware components that perform a particular function, such as a USB interface device. At an even higher level, device may refer to the function performed by an entity attached to the USB; for example, a data/FAX modem device. When used as a non-specific reference, a USB device is either a hub or a function.
High-speed	USB operation at 480 Mbit/s.
Full-speed	USB operation at 12 Mbit/s.
Low-speed	USB operation at 1.5 Mbit/s.
Endpoint	A uniquely addressable portion of a USB device that is the source or sink of information in a communication flow between the host and device.
Scatter/Gather DMA mode	Accesses discontinuous memory areas via DMA.
Buffer DMA mode	Accesses contiguous memory areas via DMA.
Slave mode	Accesses memory area through the CPU.
RX FIFO	Stores the received data.

TX FIFO	Stores the data to be transmitted.
Periodic FIFO	Stores Isochronous transfer data and Interrupt transfer data to be sent.
Non-periodic FIFO	Stores Control transfer data and Bulk transfer data to be sent.

49.3 Features

49.3.1 General Features

- USB 2.0 specification, OTG Revision 1.3 and OTG Revision 2.0 specifications
- High-Speed, Full-Speed, and Low-Speed data rates
- As a host and a device in High-Speed mode and Full-Speed mode
- Dynamic FIFO (DFIFO) sizing, each device EP/host channel can dynamically allocate a maximum of 4 KB FIFO.
- Multiple modes of memory access
 - Scatter/Gather DMA mode
 - Buffer DMA mode
 - Slave Mode
- Integrated UTMI High-Speed transceiver

49.3.2 Device Mode Features

- Endpoint 0 always present, bi-directional, consisting of EPO IN and EPO OUT
- 15 additional endpoints 1~15, configurable as IN or OUT
- Maximum of eight IN endpoints concurrently active at any time, including EPO IN
- All OUT endpoints share a single RX FIFO
- Each IN endpoint has a dedicated TX FIFO

49.3.3 Host Mode Features

- 16 host channels
- RX FIFO: shared by all periodic and non-periodic transactions
- Two TX FIFO:
 - One shared by all non-periodic transactions
 - One shared by all periodic transactions
- All of the above FIFOs share a 4 KB RAM.
- The size of each FIFO is configurable, with a maximum of 4 KB.

49.4 Functional Description

49.4.1 Controller Core and Interfaces

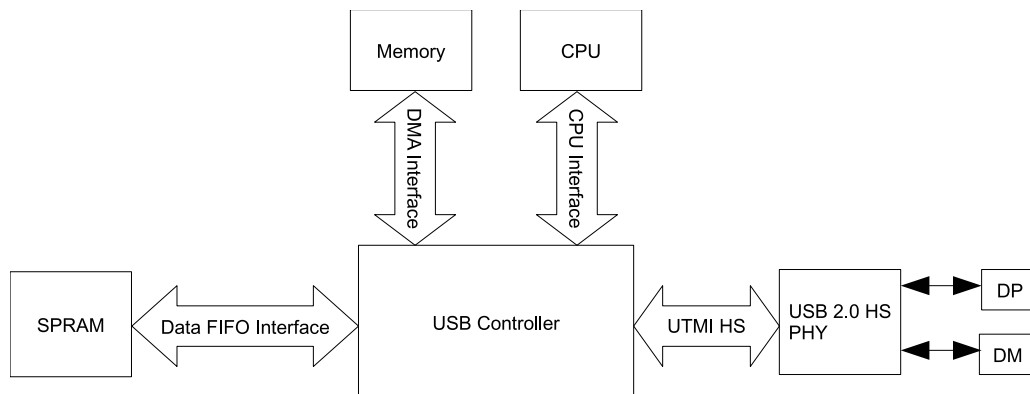


Figure 49.4-1. OTG_HS System Architecture

OTG_HS uses the same controller core as OTG_FS, i.e., USB Controller Core. The controller core has the following interfaces, see Figure 49.4-1:

- **CPU Interface**

Provides the CPU with read/write access to the controller core's various registers and FIFOs. This interface is internally implemented as an AHB slave interface. The way to access the FIFOs through the CPU interface is called Slave mode.

- **DMA Interface**

Provides the controller core's internal DMA with read/write access to system memory, e.g., fetching and writing data payloads in DMA mode. This interface is internally implemented as an AHB master interface.

- **USB 2.0 Interface (USB 2.0 HS PHY)**

This interface is used to connect the controller core to a USB 2.0 UTMI serial transceiver. With this serial transceiver, OTG_HS is able to support High-Speed data rate and Full-Speed data rate.

- **Data FIFO Interface**

The multiple FIFOs used by the controller core are not actually located within the controller core itself, but on the Single-Port RAM (SPRAM). FIFOs are dynamically sized, thus are allocated at run-time in the SPRAM. When the CPU, DMA, or the controller core attempts to read/write to FIFOs, those accesses are routed through the data FIFO RAM interface.

49.4.2 Memory Layout

Figure 49.4-2 illustrates the memory layout of the registers which are used to configure and control the USB Controller Core.

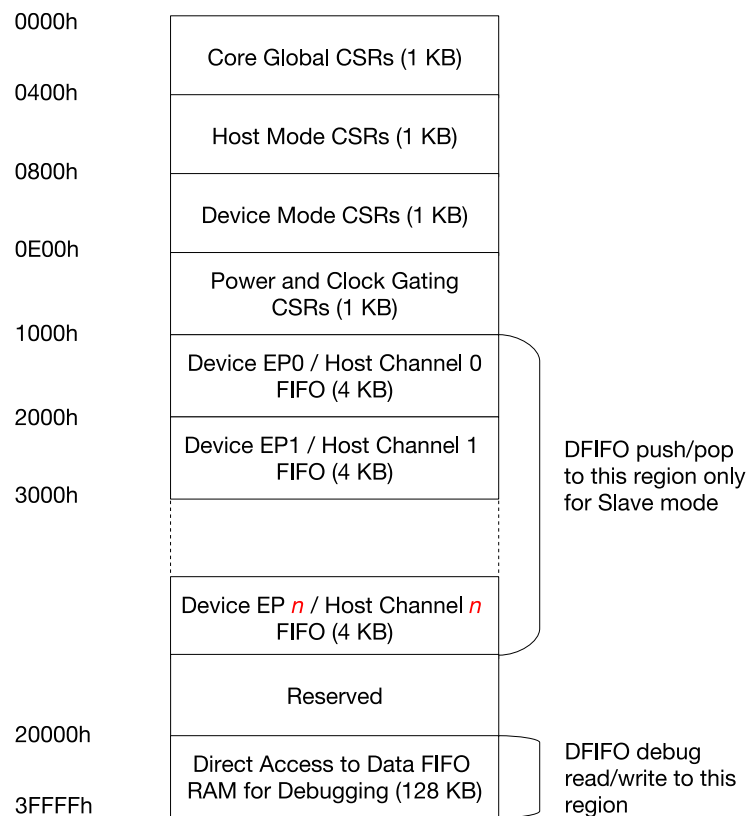


Figure 49.4-2. OTG_HS Register Layout

49.4.2.1 Control & Status Registers (CSRs)

- **Core Global CSRs**

These registers are responsible for the configuration/control/status of the global features of OTG_HS, i.e., features which are common to both Host and Device modes. These features include OTG control and system-level interrupts. Software can access these registers whilst in Host or Device modes.

- **Host Mode CSRs**

These registers are responsible for the configuration/control/status in Host mode, thus should only be accessed in Host mode. Each channel has its own set of registers within the Host mode CSRs.

- **Device Mode CSRs**

These registers are responsible for the configuration/control/status in Device mode, thus should only be accessed in Device mode. Each endpoint has its own set of registers within the Device mode CSRs.

- **Power and Clock Gating Register**

A single register controls power-down and gates various clocks.

49.4.2.2 FIFO Access

The OTG_HS makes use of multiple FIFOs to buffer the data payloads to be transmitted or to buffer the received data payloads. The number and type of FIFOs are dependent on Host or Device mode, and the number of channels or endpoints used, see Section 49.4.3. There are two ways to access the FIFOs: DMA mode and Slave mode. In Slave mode, the CPU will need to access these FIFOs by reading and writing to either the DFIFO push/pop regions or the DFIFO read/write debug region. FIFO access is governed by the

following rules:

- Read access to any address in any one of the 4 KB push/pop regions will result in a pop from the shared RX FIFO.
- Write access to a particular 4 KB push/pop region will result in a push to the corresponding endpoint or channel's TX FIFO given that the endpoint is an IN endpoint, or the channel is an OUT channel.
 - In Device mode, data is pushed to the corresponding IN endpoint's dedicated TX FIFO.
 - In Host mode, data is pushed to the non-periodic TX FIFO or the periodic TX FIFO depending on whether the channel is a non-periodic channel, or a periodic channel.
- Access to the 128 KB read/write region will result in direct read/write instead of a push/pop. This is generally used for debugging purposes only.

Note that pushing and popping data to and from the FIFOs by the CPU is only required in Slave mode. In DMA mode, the internal DMA will handle all pushing/popping of data to and from the TX and RX FIFOs.

49.4.3 FIFO and Queue Organization

The FIFOs in OTG_HS are primarily used to hold data packet payloads, i.e., the data field of USB Data packets. TX FIFOs are used to store data payloads that will be transmitted by OUT transactions in Host mode or IN transactions in Device mode. RX FIFOs are used to store received data payloads of IN transactions in Host mode or OUT transactions in Device mode. In addition to storing data payloads, RX FIFOs also store a **status entry** for each data payload. Each status entry contains information about a data payload such as channel number, byte count, and validity status. In Slave mode, status entries are also used to indicate various channel events.

The portion of SPRAM that can be used for FIFO allocation has a depth of 1024 and a width of 35 bits (32 data bits plus 3 control bits). The multiple FIFOs used by each channel (in Host mode) or endpoint (in Device mode) are allocated into the SPRAM and can be dynamically sized.

49.4.3.1 Host Mode FIFOs and Queues

The following FIFOs are used in Host mode, see Figure [49.4-3](#):

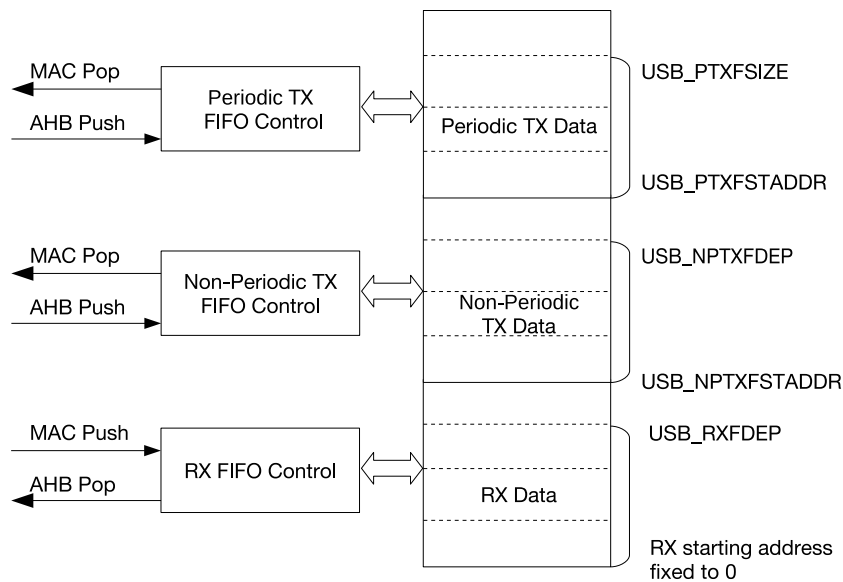


Figure 49.4-3. Host Mode FIFOs

- **Non-periodic TX FIFO:** Stores data payloads of bulk and control OUT transactions for all channels.
- **Periodic TX FIFO:** Stores data payloads of interrupt or isochronous OUT transactions for all channels.
- **RX FIFO:** Stores data payloads of all IN transactions, and status entries that are used to indicate size of data payloads and transaction/channel events such as transfer complete or channel halted.

In addition to FIFOs, Host mode also contains two request queues used to queue up the various transaction request from the multiple channels. Each entry in a request queue holds the IN/OUT channel number along with other information to perform the transaction, such as transaction type. Request queues are also used to queue other types of requests such as a channel halt request.

Unlike FIFOs, request queues are fixed in size and cannot be accessed directly by software. Rather, once a channel is enabled, requests will be automatically written to the request queue by the Host core. The order in which the requests are written into the queue determines the sequence of transactions on the USB.

Host mode contains the following request queues:

- **Non-periodic request queue:** Request queue for all non-periodic Bulk and Control channels. The queue has a depth of four entries.
- **Periodic request queue:** Request queue for all periodic Interrupt and Isochronous channels. The queue has a depth of four entries.

When scheduling transactions, hardware will execute all requests on the periodic request queue first before executing requests on the non-periodic request queue.

49.4.3.2 Device Mode FIFOs

The following FIFOs are used in Device mode, see Figure 49.4-4:

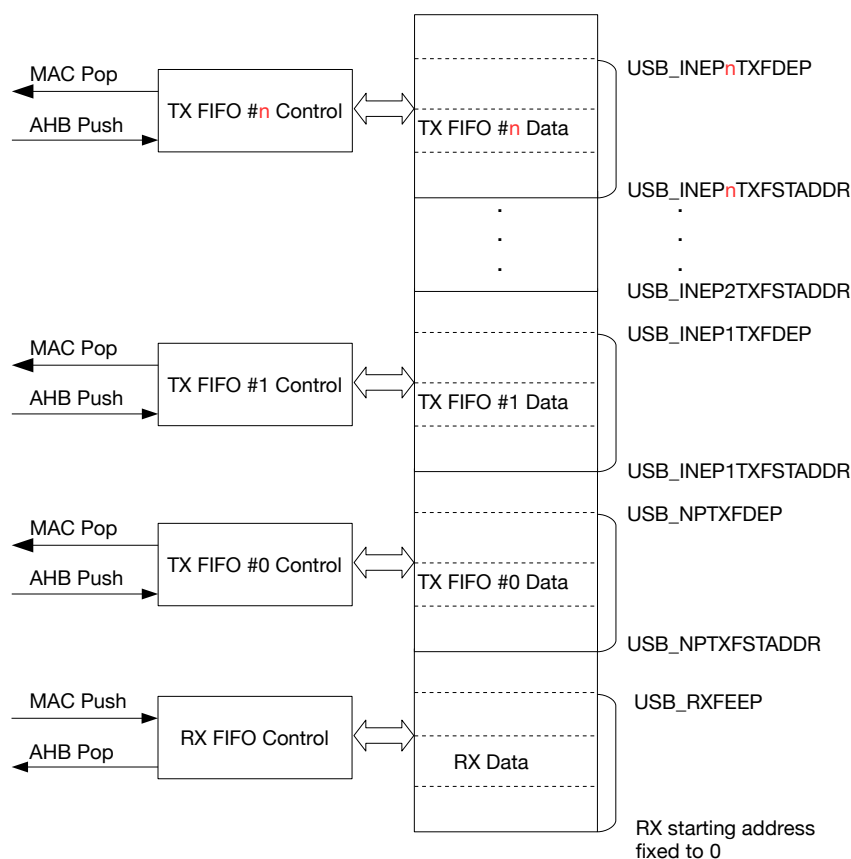


Figure 49.4-4. Device Mode FIFOs

- **RX FIFO:** Stores data payloads received in Data packet, and status entries used to indicate size of those data payloads.
- **Dedicated TX FIFO:** Each active IN endpoint will have a dedicated TX FIFO used to store all IN data payloads of that endpoint, regardless of the transaction type, both periodic and non-periodic IN transactions.

Due to the dedicated FIFOs, Device mode does not use any request queues. Instead, the order of IN transactions are determined by the Host.

49.4.4 Interrupt Hierarchy

OTG_HS provides a single interrupt line which can be routed via the interrupt matrix to one of the CPUs. The interrupt signal can be unmasked by setting USB_GLBLINTRMSK. The OTG_HS interrupt is an OR of all bits in the USB_GINTSTS_REG register, and the bits in USB_GINTSTS_REG can be unmasked by setting the corresponding bits in the USB_GINTMSK_REG register. USB_GINTSTS_REG contains system level interrupts, and also specific bits for Host or Device mode interrupts, and OTG specific interrupts. OTG_HS interrupt sources are organized as Figure 49.4-5 shows.

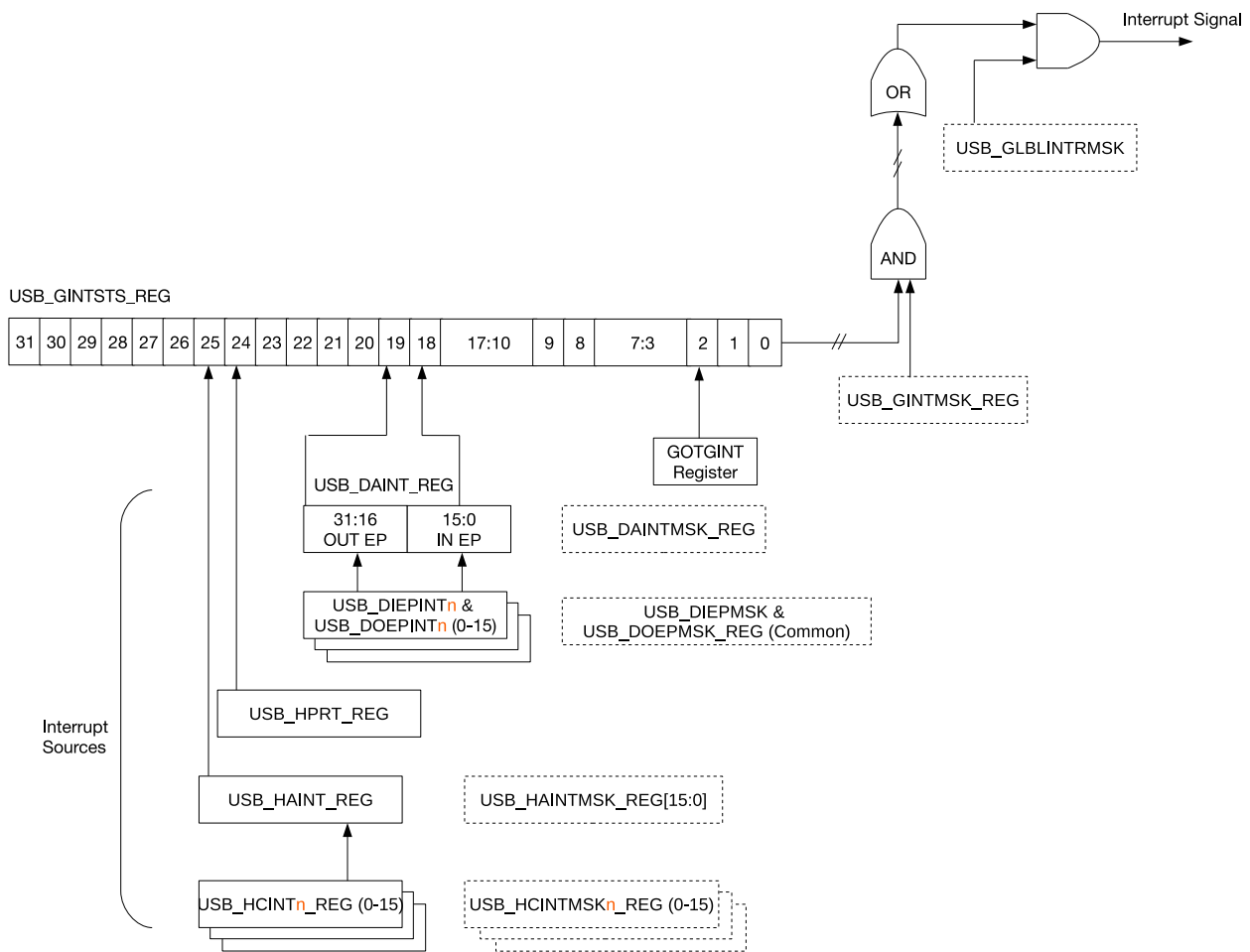


Figure 49.4-5. OTG_HS Interrupt Hierarchy

The following bits of the USB_GINTSTS_REG register indicate an interrupt source lower in the hierarchy:

- **USB_PRTINT** indicates that the Host port has a pending interrupt. The USB_HPRT_REG register indicates the interrupt source.
- **USB_HCHINT** indicates that one or more Host channels have a pending interrupt. Read the USB_HAINT_REG register to determine which channel(s) have a pending interrupt, then read the pending channel's USB_HCINT_n_REG register to determine the interrupt source.
- **USB_OEPINT** indicates that one or more OUT endpoints have a pending interrupt. Read the USB_DAINTRMSK register to determine which OUT endpoint(s) have a pending interrupt, then read the USB_DOEPINT_n_REG register to determine the interrupt source.
- **USB_IEPINT** indicates that one or more IN endpoints have a pending interrupt. Read the USB_DAINTRMSK register to determine which IN endpoint(s) are pending, then read the pending IN endpoint's USB_DIEPINT_n_REG register to determine the interrupt source.
- **USB_OTGINT** indicates an On-The-Go event has triggered an interrupt. Read the USB_GOTGINT_REG register to determine which OTG event(s) triggered the interrupt.

49.4.5 DMA Modes and Slave Mode

USB OTG supports three ways to access memory: Slave mode, Buffer DMA mode, and Scatter/Gather DMA mode.

49.4.5.1 Slave Mode

In Slave mode, all data payloads must be pushed/popped to and from the FIFOs by the CPU.

- When transmitting a packet using IN endpoints or OUT channels, the data payload must be pushed into the corresponding endpoint or channel's TX FIFO.
- When receiving a packet, the packet's status entry must first be popped off the RX FIFO by reading USB_GRXSTSP_REG. The status entry should be used to determine the length of the packet's payload in bytes. The corresponding number of bytes must then be manually popped off the RX FIFO by reading from the RX FIFO's memory region.

49.4.5.2 Buffer DMA Mode

Buffer mode is similar to Slave mode but utilizes the internal DMA to push and pop data payloads to the FIFOs.

- When transmitting a packet using IN endpoints or OUT channels, the data payload's address in memory should be written to the USB_HCDMA n _REG (in Host mode) or USB_DIEPDMA n _REG (in Device mode) registers. When the endpoint or channel is enabled, the internal DMA will push the data payload from memory into the TX FIFO of the channel or endpoint.
- When receiving a packet using OUT endpoints or IN channels, the address of an empty buffer in memory should be written to the USB_HCDMA n _REG (in Host mode) or USB_DOEPDMA n _REG (in Device mode) registers. When the endpoint or channel is enabled, the internal DMA will pop the data payload from RX FIFO into the buffer.

49.4.5.3 Scatter/Gather DMA Mode

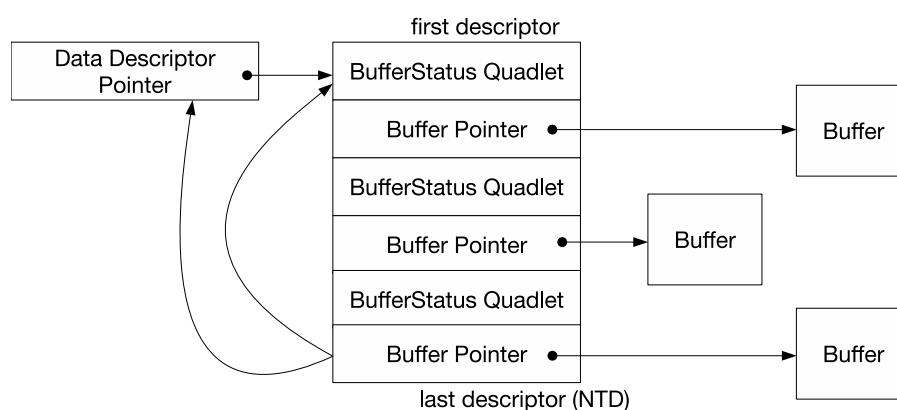


Figure 49.4-6. Scatter/Gather DMA Descriptor List

In Scatter/Gather DMA mode, buffers containing data payloads can be scattered throughout memory. Each endpoint or channel will have a contiguous DMA descriptor list, where each descriptor contains a 32-bit

pointer to the data payload or buffer and a 32-bit buffer descriptor (BufferStatus Quadlet). The data payloads and buffers can correspond to a single transaction (i.e., < 1 MPS bytes) or an entire transfer (> 1 MPS bytes). MPS: maximum packet size. The list is implemented as a ring buffer meaning that the DMA will return to the first entry when it encounters the last entry on the list.

- When transmitting a transfer/transaction using IN endpoints or OUT channels, the DMA will gather the data payloads from the multiple buffers and push them into a TX FIFO.
- When receiving a transfer/transaction using OUT endpoints or IN channels, the DMA will pop the received data payloads from the RX FIFO and scatter them to the multiple buffers pointed to by the DMA list entries.

49.4.6 Transaction and Transfer Level Operation

In either Host or Device mode, communication can operate either at the transaction level or the transfer level.

49.4.6.1 Transaction and Transfer Level Operation in DMA Mode

When operating at the transfer level in DMA Host mode, software is interrupted only when a channel has been halted. Channels are halted when their programmed transfer size has completed successfully, has received a STALL, or if there are excessive transaction errors, e.g., 3 consecutive transaction errors. In DMA Device mode, all errors are handled by the controller core itself.

When operating at the transaction level in DMA mode, the transfer size is set to the size of one data packet, either a maximum packet size or a short packet size.

49.4.6.2 Transaction and Transfer Level Operation in Slave Mode

When operating at the transaction level in Slave Mode, transfers are handled one transaction at a time. Each data payload should correspond to a single data packet, and software must determine whether a retry of the transaction is necessary based on the handshake response received on the USB, e.g., ACK or NAK.

The following table describes transaction level operation in Slave mode for both IN and OUT transactions.

Table 49.4-1. IN and OUT Transactions in Slave Mode

Host Mode	Device Mode
OUT Transactions	

Host Mode	Device Mode
1. Software specifies the size of the data packet and the number of data packets (1 data packet) in the USB_HCTSIZ_n_REG register, enables the channel, then copies the packet's data payload into the TX FIFO. 2. When the last DWORD of the data payload has been pushed, the controller core will automatically write a request into the appropriate request queue. 3. If the transaction is successful, the USB_XFERCOMPL interrupt will be generated. If the transaction is unsuccessful, an error interrupt, e.g., USB_H_NACK_n will occur.	1. Software specifies the expected size of the data packet (1 MPS) and the number of data packets (1 data packet) in the USB_DOEPSIZ_n_REG register. Once the endpoint is enabled, it will wait for the host to transmit a packet to it. 2. The received packet will be pushed into the RX FIFO along with a packet status entry. 3. If the transaction is unsuccessful, e.g., due to a full RX FIFO, the endpoint will automatically NAK the incoming packet.
IN Transactions	
1. Software specifies the expected size of the data packet and the number of packets (1 data packet) in the USB_HCTSIZ_n_REG register, then enables the channel. 2. The controller core will automatically write a request into the appropriate request queue. 3. If the transaction is successful, the received data along with a status entry should be written to the RX FIFO. If the transaction is unsuccessful, an error interrupt, e.g., USB_H_NACK_n will occur.	1. Software specifies the size of the data packet and the number of data packets (1 data packet) in the USB_DIEPTSIZ_n_REG register. Once the endpoint is enabled, it will wait for the host to read the packet. 2. When the packet has been transmitted, the USB_XFERCOMPL interrupt will be generated.

When operating at the transfer level in Slave mode, one or more transaction-level operations can be pipelined thus being analogous to transfer level operation in DMA mode. Within pipelined transactions, multiple packets of the same transfer can be read/written from the FIFOs in single instance, thus preventing the need for interrupting the software on a per-packet basis.

Operating on a transfer level in Slave mode is similar to operating on the transaction-level, except the transfer size and packet count for each transfer in the USB_HCTSIZ_n_REG or USB_DOEPSIZ_n_REG register will need to be set to reflect the entire transfer. After the channel or endpoint is enabled, multiple data packets worth of payloads should be written to or read from the TX or RX FIFOs respectively (given that there is enough space or enough data).

49.4.7 OTG

Internal ID/VBUS is left floating in OTG_HS, thus SRP/HNP is not supported. But OTG_HS still supports Dual-Role-Devices (DRD), i.e, it can be used both as a host and a device. Software can force setting this

controller to work as host or device by configuring ForceDevMode or ForceHstMode in register USB_GUSBCFG, regardless of the value of ID pin.

49.5 Registers

The catalog and comprehensive specifications of USB OTG registers are subject to a Non-Disclosure Agreement (NDA) as mandated by the IP provider. To obtain support information for a particular register, please contact Espressif Technical Support Team via [Technical Inquires](#).